PROJECT DOCUMENTATION

AI BLOGNEST - GENERATE HIGH-QUALITY BLOG CONTENT USING AI.

INTRODUCTION:

Project Title: AI BlogNest – Generate high-quality blog content using AI.

Team Size: 5

Team Leader: Mercy Cuety T – 24csmercycuety056@gmail.com

Team Members:

- Monika M – 24csmonikam057@gmail.com
- Monisha S – 24csmonisha.s058@gmail.com
- Monishri V – 24csmonishri059@gmail.com
- Nathiya Devi S – 24csnathiyadevi060@gmail.com

DESCRIPTION:

BlogNest is a web-based content management platform developed for creating, managing, and organizing blogs efficiently. The backend is built using Node.js and Express.js, with MongoDB and Mongoose used for database management.

The system integrates Google Gemini AI to provide features such as AI-assisted blog generation, FAQ support, weather insights, and fitness recommendations. These features help users create useful content quickly with less manual effort.

For security, the application uses JWT authentication, role-based access control, and bcrypt.js for password protection. It also includes blog management, categories, tags, comments, media handling, and search functionality.

The modular architecture makes the system easy to maintain and extend. By combining blogging features with Generative AI, BlogNest provides a smart and efficient platform for modern content creation and management.

SCENARIO-BASED CASE STUDY:

BACKGROUND:

Rohan is a content creator who manages several blogs. She finds it difficult to organize posts, handle categories and tags, manage comments, and monitor reader activity through separate tools.

PROBLEM:

- ❖ Managing multiple blog drafts is difficult.
- ❖ Organizing categories and tags takes time.
- ❖ Tracking post views and reader engagement is not simple.
- ❖ Comment moderation and content approval require manual effort.
- ❖ Managing media files and user permissions is complicated.

SOLUTION:

BlogNest provides a centralized backend system for managing blog content and users. It offers secure APIs for blog creation, content organization, media management, comments, publishing, and role-based access.

USAGE:

- **AUTHOR:** Creates and manages blog posts, organizes drafts, and adds media.

- **EDITOR/ADMIN:** Reviews posts, manages publishing status, moderates comments, and controls user permissions.

- **READER:** Views published blogs and can add comments.

OUTCOME:

BlogNest makes content management more organized and efficient. Data validation and security controls help protect the system, while role-based permissions ensure that users can access only the features assigned to them.

1. SOFTWARE REQUIREMENTS:

- Operating System: Windows 10/11, macOS, or Linux
- Node.js: Version 16 or above
- npm: Version 8 or above
- Express.js: For developing REST APIs
- MongoDB: For storing users, blogs, categories, comments, and related data
- Postman: For testing and verifying APIs
- Visual Studio Code: For development

2. HARDWARE REQUIREMENTS:

- Processor: Intel Core i5 / AMD Ryzen 5 or equivalent
- RAM: Minimum 8 GB
- Storage: At least 1 GB of free space

Epic:-1 PROJECT ARCHITECTURE

The BlogNest application follows a layered backend architecture where client requests are processed through different components before reaching the database. Each layer performs a specific function to provide secure and organized API operations.

COMPONENTS AND THEIR FUNCTIONS:

1. USER INTERFACE / API CLIENT:
Tools such as Postman send API requests to the required endpoints and receive responses in JSON format.

2. EXPRESS SERVER:
The Express server receives requests, manages HTTP methods such as GET, POST, PUT, and DELETE, processes JSON data, and handles CORS.

3. AUTHENTICATION AND AUTHORIZATION:
This layer verifies JWT tokens and checks the user's role before allowing access to protected resources.

4. BUSINESS LOGIC SERVICES:
These modules manage important operations such as blog creation, editing, approval, scheduling, publishing, comment management, and reports.

5. DATABASE LAYER:
Mongoose connects the application with MongoDB and manages data storage, validation, and database operations.

6. AI INTEGRATION LAYER:
The AI service connects BlogNest with Google Gemini AI to provide features such as blog generation, content summarization, FAQ assistance, and other intelligent content services.

7. AI BLOG GENERATOR:
This module uses the AI service to automatically generate blog content, helping users prepare drafts faster and reducing manual writing effort.

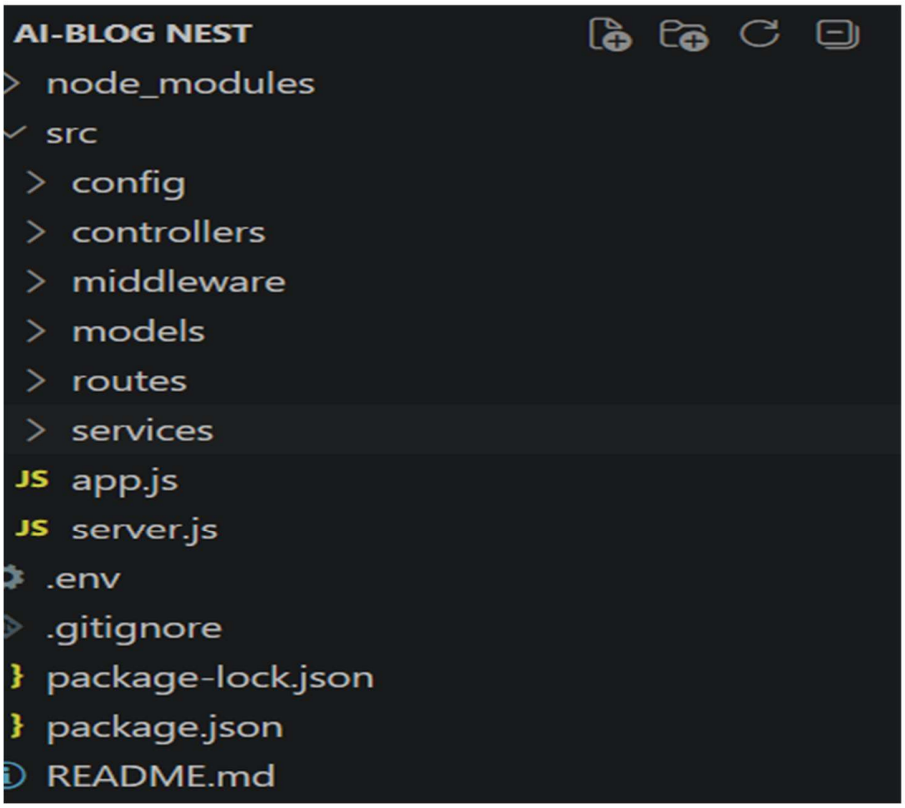
KEY FEATURES:

- Role-Based Access: Controls system access based on user roles and protects restricted operations.
- Blog CRUD Management: Allows users to create, view, update, and delete blog posts.
- Categories and Tags: Helps organize blogs and makes content easier to search and filter.
- Comment Management: Provides options to review and manage comments based on their status.
- AI Blog Generation: Uses AI technology to create structured blog content from user-provided topics.
- AI Content Summarization: Generates short summaries from longer blog content for quick understanding.

EPIC2: PROJECT SETUP AND CONFIGURATION:

CREATING PROJECT FOLDER:

1) Create a new folder with your <project name>.
2) Inside that folder create one new folders.
3) Name another one as Server.
4) Now open that folder in VS Code.

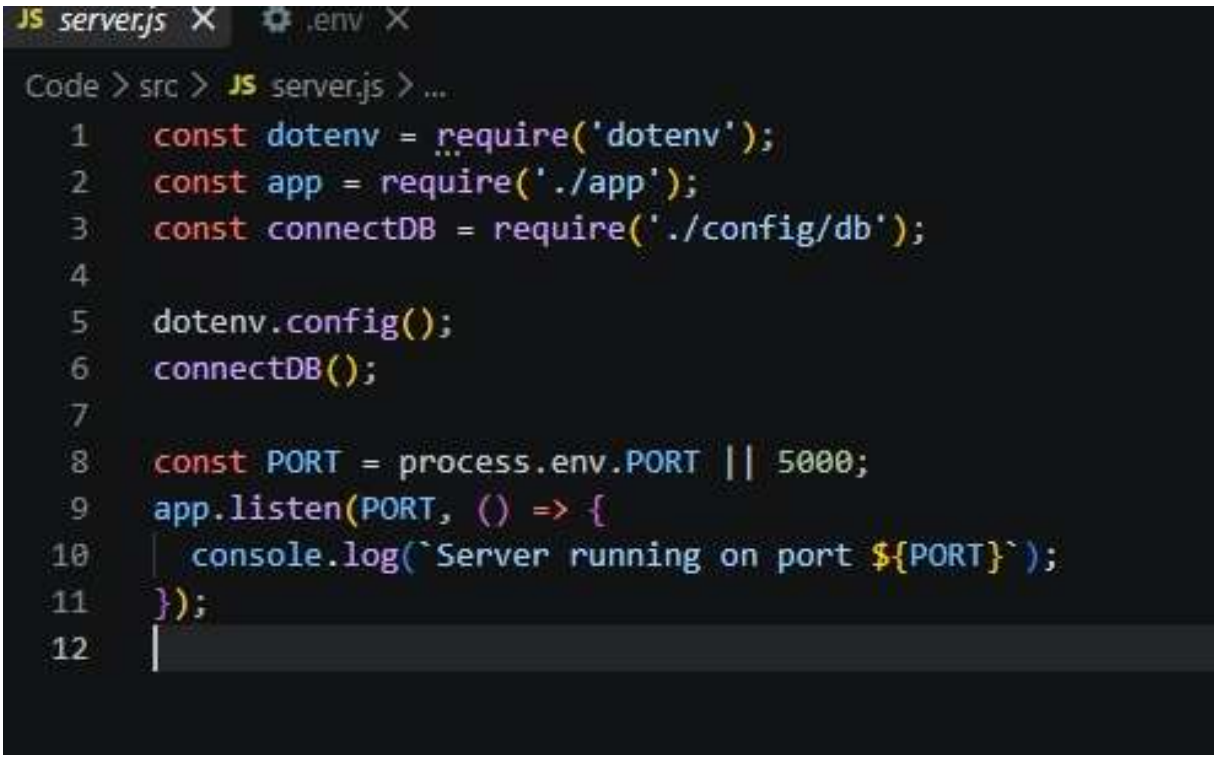


SERVER SETUP (NPM INIT):

- Open Server folder in terminal of VScode.
  - npm init -y
- Create files:
  - server.js
- Create folders:
  - models
  - controllers
  - routes

Epic 3: Backend Development:

SERVER.JS:



APP.JS



```
JS app.js ✕  .env ✕
Code > src > JS app.js > ...
1   const express = require('express');
2   const cors = require('cors');
3   const morgan = require('morgan');
4   const authRoutes = require('./routes/authRoutes');
5   const blogRoutes = require('./routes/blogRoutes');
6   const aiRoutes = require('./routes/aiRoutes');
7   const errorMiddleware = require('./middleware/errorMiddleware');
8
9   const app = express();
10
11  app.use(cors());
12  app.use(express.json());
13  app.use(morgan('dev'));
14
15  app.use('/api/auth', authRoutes);
16  app.use('/api/blogs', blogRoutes);
17  app.use('/api/ai', aiRoutes);
18
19  app.use(errorMiddleware);
20
21  app.get('/', (req, res) => {
22    res.json({ message: 'AI BlogNest API is running' });
23  });
24
25  module.exports = app;
26
```

EPIC4: DATABASE CONFIGURATION:

DB CONNECTION:-



```
JS db.js ✕  .env ✕
Code > src > config > JS db.js > ...
1   const mongoose = require('mongoose');
2
3   const connectDB = async () => {
4     const mongoUri = process.env.MONGO_URI;
5     if (!mongoUri) {
6       console.error('MongoDB connection error: MONGO_URI is not defined in environment variables');
7       process.exit(1);
8     }
9
10    try {
11      const conn = await mongoose.connect(mongoUri, {
12        useNewUrlParser: true,
13        useUnifiedTopology: true,
14      });
15      console.log(`MongoDB connected: ${conn.connection.host}`);
16    } catch (error) {
17      console.error(`MongoDB connection error: ${error.message}`);
18      process.exit(1);
19    }
20  };
21
22  module.exports = connectDB;
23
```

1. CONFIGURE MONGODB:

- Install Mongoose.
  - In your Node.js project directory, run:
  - npm install mongoose
- Create database connection.
- Create Schemas & Models.

CREATE DATABASE CONNECTION:

Now, make sure the database is connected before performing any of the actions through the backend. The connection code looks similar to the one provided below.

MONGOOSE SCHEMA DEFINITIONS:

1. IDENTITY RESOURCE DOCUMENT SCHEMA (schemas/userModel.js):

Enforces field criteria validations on incoming database creations:

User.js .env

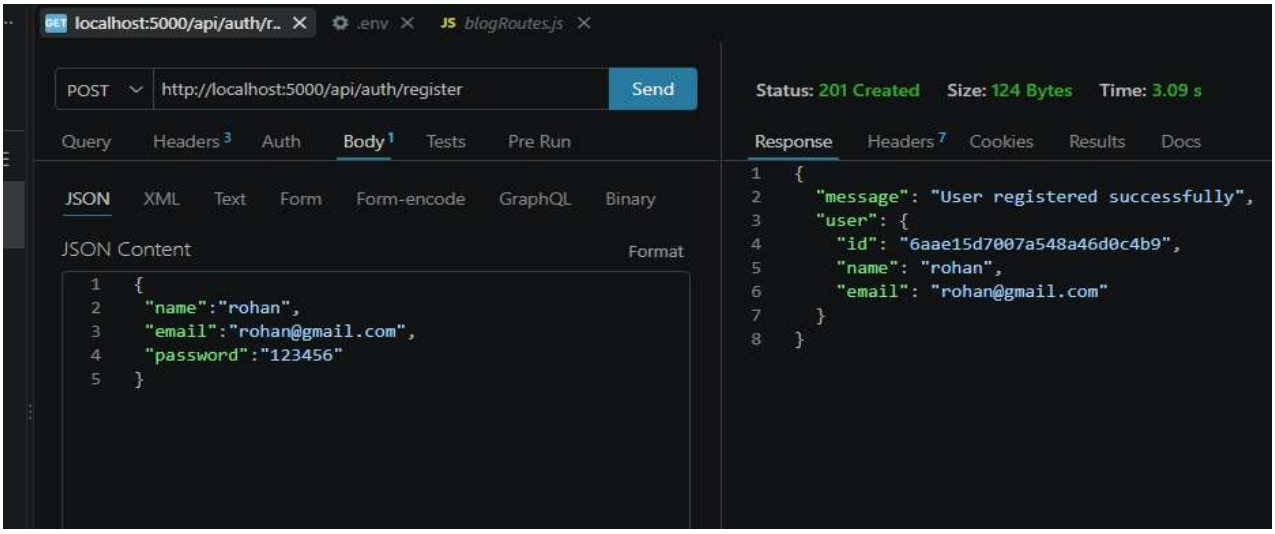
Code > src > models > User.js > ...

```
1   const mongoose = require('mongoose');
2
3   const userSchema = new mongoose.Schema(
4     {
5       name: {
6         type: String,
7         required: [true, 'Name is required'],
8         trim: true,
9       },
10      email: {
11        type: String,
12        required: [true, 'Email is required'],
13        unique: true,
14        lowercase: true,
15        trim: true,
16      },
17      password: {
18        type: String,
19        required: [true, 'Password is required'],
20        minlength: 6,
21      },
22    },
23    {
24      timestamps: true,
25    }
26  );
27
28  module.exports = mongoose.model('User', userSchema);
```

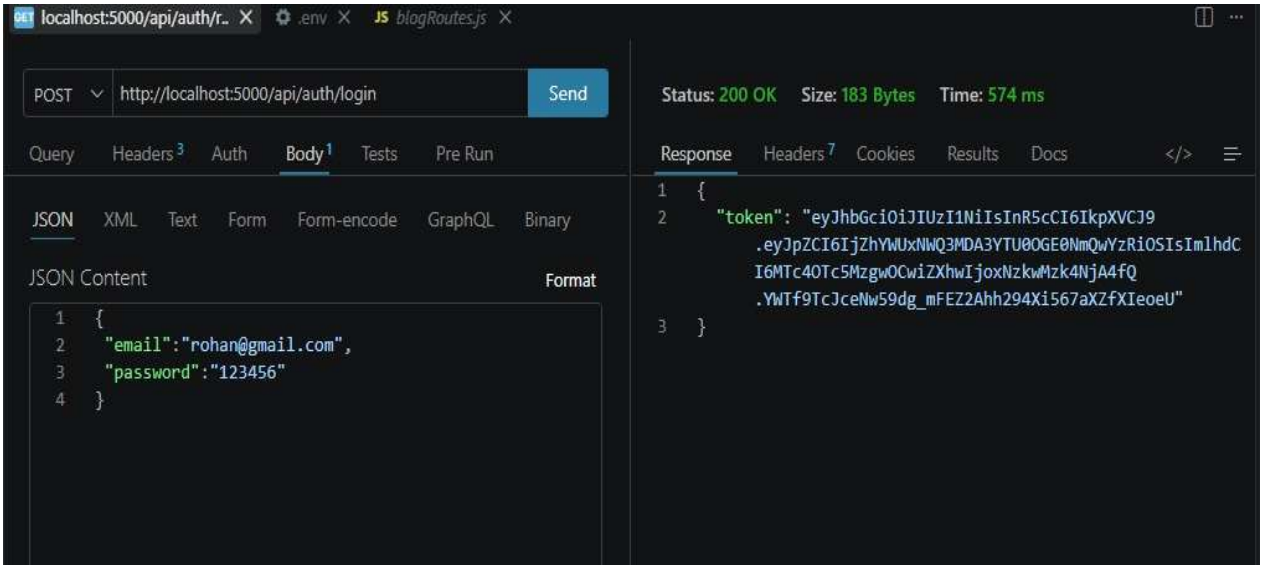
EPIC 5: PROJECT EXECUTIONS:

- Install Postman or ThunderClient (VS Code extension).
- Start the backend server:
  “npm start”

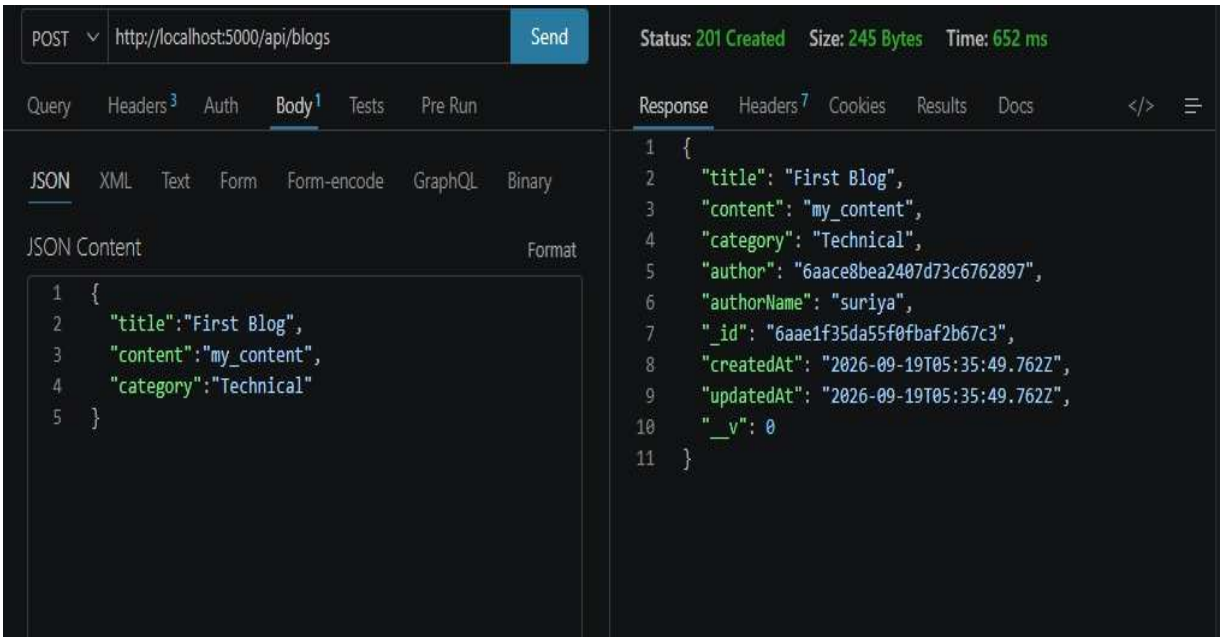
API TESTING:
USER REGISTRATION :



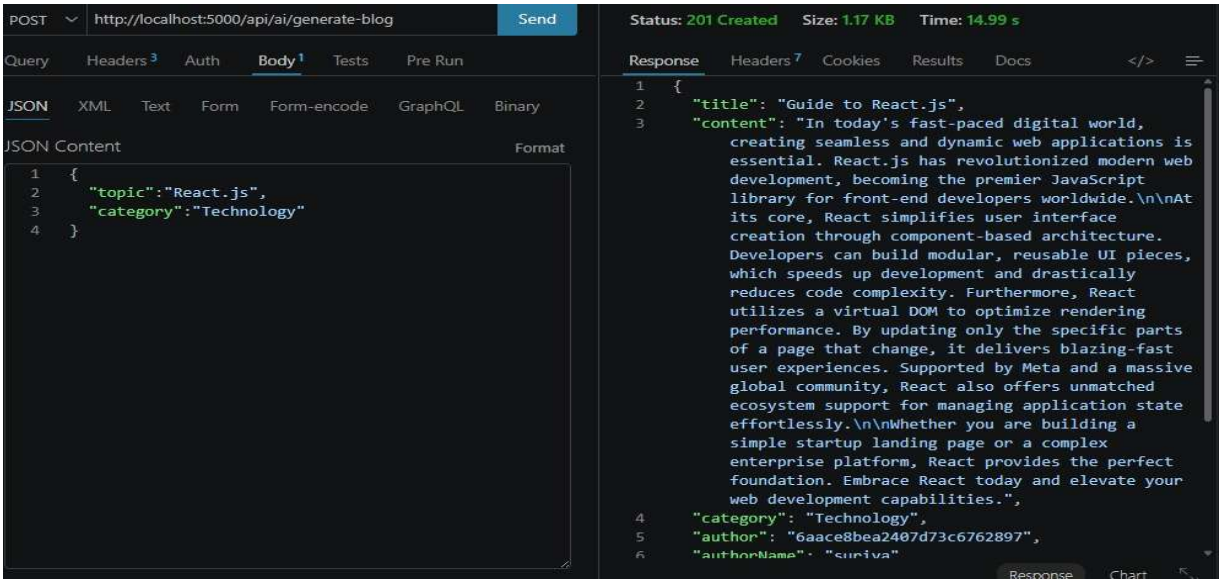
USER LOGIN:



CREATE BLOGS :



GENERATE AI BLOGS :



DEMO VIDEO: Link